



[View, add and edit your notes in the app](#)

DBMS 05 | Transaction & Concurrency Control | CS & IT | GATE 2024 FastTrack Batch

Generated on October 23, 2024

Summary

AI Notes

AI Slides

Text Notes

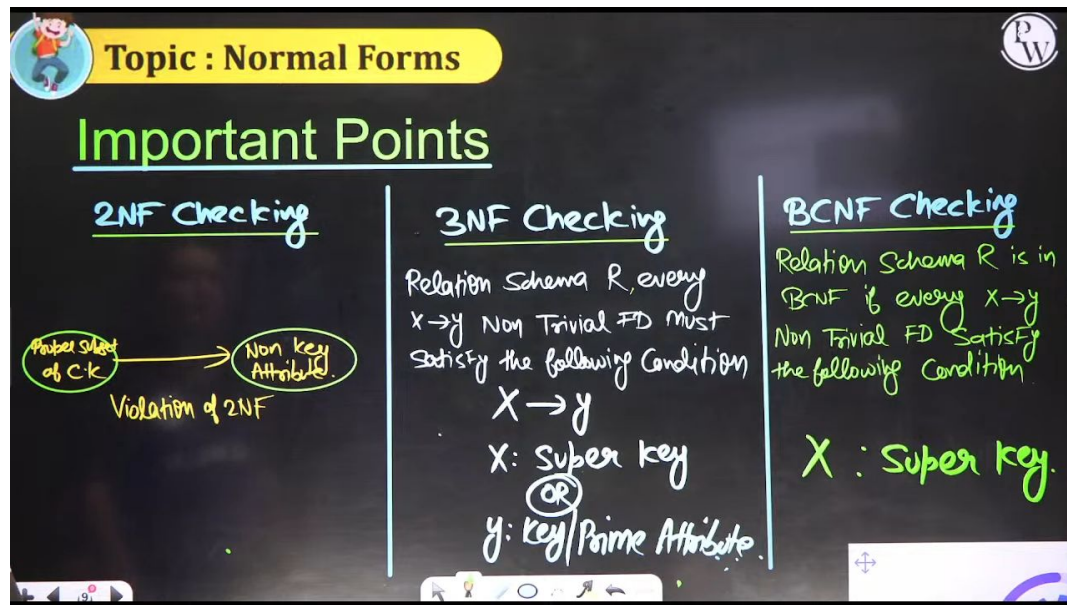
Screenshots

✦ 0

✦ 0

📄 2

📷 36



The image shows a handwritten note on a blackboard background. At the top left, there is a small cartoon character and a yellow banner that says "Topic : Normal Forms". In the top right corner, there is a small circular logo with the letters "PW". The main title "Important Points" is written in green. Below it, there are three columns of text, each with a heading in green: "2NF Checking", "3NF Checking", and "BCNF Checking".

2NF Checking

Proper subset of CK → Non key Attribute.
Violation of 2NF

3NF Checking

Relation Schema R, every $X \rightarrow Y$ Non Trivial FD must satisfy the following Condition

$X \rightarrow Y$

X: Super key
OR
Y: Key/Prime Attribute

BCNF Checking

Relation Schema R is in BCNF if every $X \rightarrow Y$ Non Trivial FD Satisfy the following Condition

X : Super key.

▶ 10:38



Important Points :

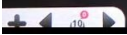
If a Relation R has Only One Candidate key then Relation R always is in 1NF But May or May Not in 2NF, 3NF, 4NF, BCNF.

⑧ $R(ABCDEF) [AB \rightarrow C, C \rightarrow D, D \rightarrow F, B \rightarrow E]$

Candidate key = $\{AB\}$

Non key Attribute = {C, D, E, F}

$B \rightarrow E$
 Proper Subset of one CK Non key Attribute } Not in 2NF



Topic : Normal Forms



Important Points :

If a Relation R, all Candidate keys are simple Candidate key (no Composite key) then R always in 2NF But may not in 3NF & 4NF

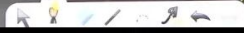
② $R(ABCDE)$ ($A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow E, C \rightarrow \underline{A}$)

Candidate key = [A, C, B]

R is in 2NF But Not in 3NF. ($\because D \rightarrow E$) (or)

$D \rightarrow E$; D is Not Super Key
(OR) E is Non Prime Attribute

$C \rightarrow D, D \rightarrow E$ by PK
Non key transitively determine





Topic : Normal Forms



Important Points about 3NF.

If in a Relation R, all attributes are key/prime Attribute.
(No Non Prime Attribute) then Relation R always is in 3NF,
But May or May Not in BCNF.

ex) R(ABCDE) [A → B, B → C, C → D, D → A]

Candidate keys = [AE, BE, CE, DE]

key/Prime Attribute = {A, B, C, D, E}

all attribute
are prime
Attribute

R is in 3NF But Not in BCNF.

X → Y
A → B
B → C
C → D
D → A

X: Not Super key.

▶ 28:25



Topic : Normal Forms



Important Points about BCNF

If a Relation R is 3NF & Every Candidate key is
a Simple Candidate key (No Composite key) then Relation R
always is in BCNF.

ex) R(ABCD) [A → B, B → C, C → D, D → A]

Candidate key = [A, B, C, D]

& key/Prime Attribute = {A, B, C, D}

here All attributes are
key Attribute so

R is in 3NF & all keys are simple CK so R is in BCNF.

X → Y
A → B
B → C
C → D
D → A

X: Super key

BCNF

▶ 33:43



Topic : Normal Forms



Important Points about BCNF

Binary Relation (Relation with 2 Attributes) is always in BCNF.

② R(AB) [A → B]

Candidate key = {A}

A → B; A is super key.

So R is in BCNF.

R(AB) [B → A]

Candidate key = {B}

B → A; B is super key

∴ R is in BCNF

R(AB) [A → B, B → A]

Candidate key = {A, B}

A → B; A is super key

B → A; B is super key

∴ R is in BCNF.

▶ 37:44



Topic : Normal Forms



Important Points

Design Goal	1NF	2NF	3NF	BCNF
0-1 Redundancy	X	X	X	✓ But suffers from multivalued FD $X \twoheadrightarrow Y$
Lossless Join Decomposition	YES	YES	YES	YES
Dependency Preserving	YES	YES	YES	May / May Not

▶ 39:59

[MCQ] **[2 Marks]** 

#Q. The relation scheme student Performance (name, courseNo, rollNo, grade) has the following functional dependencies:

- ✓ name, courseNo → grade ① Name Course No.
- ✓ rollNo, courseNo → grade ② Roll No Course No.

3NF But Not in BCNF

name → rollNo; Name is Not Super Key
rollNo → name: RollNo is Not Super Key

The highest normal form of this relation scheme is

Key / Prime Attribute = {name, RollNo, Course No.}

☒ A 2 NF
☐ B 3 NF
☐ C BCNF
☐ D 4 NF

Ans (B)

▶ 47:23

f

▶ 47:23

Doubt : name, courseNo -> rollno, but its a partial dependency then why 3 NF as it fails 2NF

▶ 47:23

[MCQ] **[2 Marks]** 

#Q. Relation R has eight attributes ABCDEFGH. Fields of R contain only atomic values

$F = \{CH \rightarrow G, A \rightarrow BC, B \rightarrow CFH, E \rightarrow A, F \rightarrow E\}$ is a set of functional dependencies (FDs) so that F^+ is exactly the set of FDs that hold for R. The relation R is

2013: 4marks

2 Marks

CK
AD, ED
FD, BD

Nonkey Attribute = {C, E, G}

☒ A in 1 NF, but not in 2 NF.
☐ B in 2 NF, but not in 3 NF.
☐ C in 3NF, but not in BCNF.
☐ D in BCNF.

$B \rightarrow C, F \rightarrow E, F \rightarrow G$
 Violation of 2NF.

▶ 1:03:47

[MCQ]

[2 Marks]



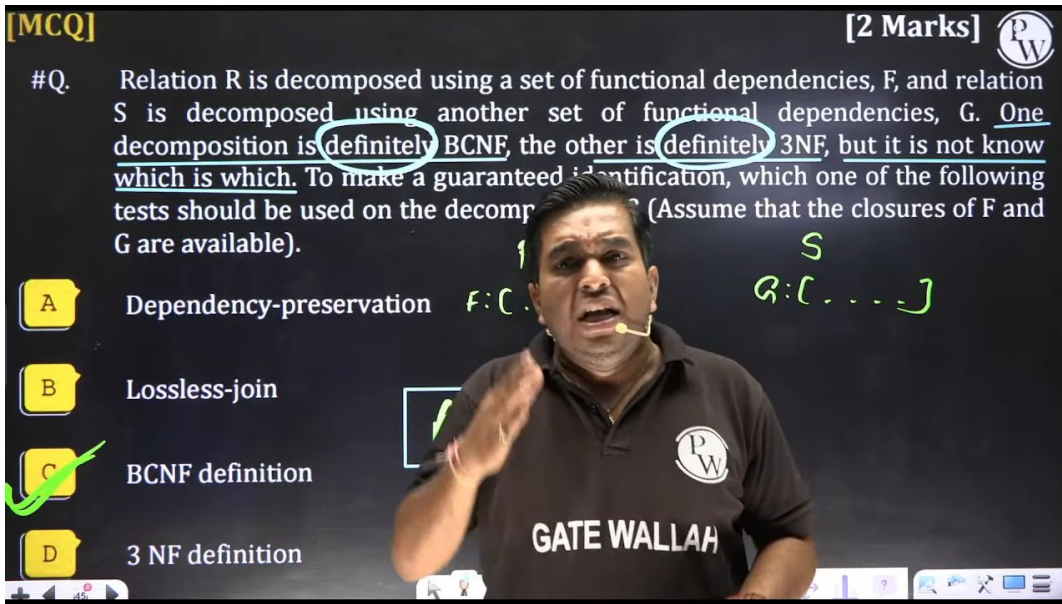
#Q. Relation R is decomposed using a set of functional dependencies, F, and relation S is decomposed using another set of functional dependencies, G. One decomposition is definitely BCNF, the other is definitely 3NF, but it is not known which is which. To make a guaranteed identification, which one of the following tests should be used on the decomposition? (Assume that the closures of F and G are available).

A. Dependency-preservation

B. Lossless-join

☒ C. BCNF definition

D. 3 NF definition



▶ 1:12:03



Topic : ACID Properties

□ A transaction is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure:

- A. Atomicity
- C. Consistency
- I. Isolation
- D. Durability



▶ 1:17:53



Topic : Transaction Concept

- ❑ A transaction is a unit of program execution that accesses and possibly updates various data items.
- ❑ E.g. Transaction to transfer Rs 500 from account A to account B:
 1. read (A)
 2. $A := A - 500$
 3. write(A)
 4. read(B)
 5. $B := B + 500$
 6. write(B)

▶ 1:25:08



Topic : ACID Properties

- ❑ **Atomicity:** Either all operations of the transaction are properly reflected in the database or none are.
- ❑ **Consistency:** Execution of a transaction in isolation preserves the consistency of the database.
- ❑ **Isolation:** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
 - ❖ That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.
- ❑ **Durability:** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

▶ 1:25:39

① Atomicity : [FULL or None]

- Power failure
- Slow crash
- H/w crash
- N/w Issue & etc.

Due to Any of these Reason if Atomicity is getting failed, then Recovery Management Component are there.

Note if Transaction is failed before Commit then Recovery Management Component ROLLBACK (UNDO ALL modification) the Transaction. with the help of Transaction Log's.

Log file : Contain All activity of the Transaction.

② Cstart $T_1, X. 5, 7$
 ↑ ↑
 old value new value

▶ 1:28:39

② Consistency

Before.

A: 2000
+ B: 4000
6000

500 Rs

After

A: 1500
+ B: 4500
6000

▶ 1:32:02

Chg Fees: Bank Account

- CHEQUE Book
- ATM
- UPS
- Mobile Banking
- By Cash
- By Internet Banking.

Isolation: Concurrent execution
of 2 @ More
transaction should be
equal to any Serial
Schedule.

▶ 1:39:20

④ Durability: Any change in the Database must persist for long period of time (if successfully executed)

Database must be OR able to Recover under Any Case of failure.

▶ 1:44:28



Topic : Transaction State

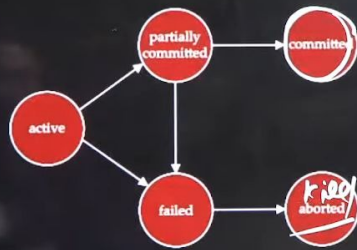


- ❑ Active: the initial state; the transaction stays in this state while it is executing.
- ❑ Partially committed: after the final statement has been executed.
- ❑ Failed: after the discovery that normal execution can no longer proceed.
- ❑ Aborted: after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - ❖ Restart the transaction
 - Can be done only if no internal logical error
 - ❖ Kill the transaction
- Committed: After successful completion.

▶ 1:44:50



Topic : Transaction State (Cont.)



▶ 1:45:02



Topic : Schedules



- ❑ **Schedule:** a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed.
 - ❖ A schedule for a set of transactions must consist of all instructions of those transactions
 - ❖ Must preserve the order in which the instructions appear in each individual transaction.
- ❑ A transaction that successfully completes its execution will have a commit instructions as the last statement
 - ❖ By default transaction assumed to execute commit instruction as its last step
- ❑ A transaction that fails to successfully complete its execution will have an abort instruction as the last statement.

▶ 1:45:03

SCHEDULE

Schedule is a Time Order Sequence of Two (or) More Transaction.

① **Serial Schedule**: Execution of one Transaction Completely then another Transaction Starts.

② **Non Serial Schedule**: Intertwined execution of Two (or) More Transaction.

▶ 1:51:10



Topic : Serial Schedule



- ❑ After Commit of one transaction, begins (Start) another transaction.
- ❑ Number of possible serial Schedules with 'n' transactions is "n!"
- ❑ The execution sequence of Serial Schedule always generates consistent result.

Example

S : $R_1(A)$ $W_1(A)$ Commit (T_1) $R_2(A)$ $W_2(A)$ commit (T_2).

▶ 1:51:35



Topic : Serial Schedule



Advantage

- Serial Schedule always produce correct result (integrity guaranteed) as no resource sharing.

Disadvantage

- Less degree of concurrency.
- Through put of system is low.
- It allows transactions to execute one after another.

▶ 1:53:58



Topic : Schedules



Let T_1 transfer 100 Rs from A to B, and T_2 transfer 10% of the balance from A to B.

Schedule 1

T_1	T_2
read (A) $A=2000$	
$A := A - 100$	
write (A) $A=1900$	
read (B) $B=3000$	
$B := B + 100$	
write (B) $B=3100$	
commit	
	read (A) $A=1900$
	temp := $A * 0.1$ temp = 190
	$A := A - \text{temp}$
	write (A) $A=1710$
	read (B) $B=3100$
	$B := B + \text{temp}$
	write (B) $B=3290$
	Commit

Handwritten notes for Schedule 1:
 $A: 2000$
 $+ B: 3000$
 $= 5000$
 $A: 1710$
 $+ B: 3290$
 $= 5000$
 Consistent
 $S_1 < T_1$

Schedule 2

T_1	T_2
read (A) $A=2000$	
$A := A - 100$	
write (A) $A=1900$	
read (B) $B=3000$	
$B := B + 100$	
write (B) $B=3100$	
commit	
	read (A) $A=2000$
	temp := $A * 0.1$ temp = 200
	$A := A - \text{temp}$
	write (A) $A=1800$
	read (B) $B=3000$
	$B := B + \text{temp}$
	write (B) $B=3200$
	Commit

Handwritten notes for Schedule 2:
 $A: 2000$
 $+ B: 3000$
 $= 5000$
 $A: 1700$
 $+ B: 3300$
 $= 5000$
 Consistent
 $A=1800$
 $B=3200$
 $A=1700$
 $B=3300$
 $S_2 < T_2, T_1 >$

▶ 2:00:35

Note Serial Schedule (All Serial Schedule [n!])
are always Consistent.

▶ 2:01:10

Topic : Schedules

$$\begin{array}{r} A: 2000 \\ + B: 3000 \\ \hline 5000 \end{array}$$

Schedule 3

T_1	T_2
read (A) A := A - 100 write (A)	read (A) temp := A * 0.1 A := A - temp write (A)
read (B) B := B + 100 write (B) commit	read (B) B := B + temp write (B) Commit

C_1

$$\begin{array}{r} A: 2000 \\ + B: 3000 \\ \hline 5000 \end{array}$$

Schedule 4

T_1	T_2
read (A) A := A - 100	read (A) temp := A * 0.1 A := A - temp write (A)
write (A) read (B) B := B + 100 write (B) commit	read (B) B := B + temp write (B) Commit

C_2

▶ 2:01:14

① $\langle T_1, T_2 \rangle$ T_1 followed by T_2

② $\langle T_2, T_1 \rangle$ T_2 followed by T_1

③ 3 Transaction $\Rightarrow 3! = 6$ Serial Schedule

$\langle T_1, T_2, T_3 \rangle$
 $\langle T_1, T_3, T_2 \rangle$
 $\langle T_2, T_1, T_3 \rangle$
 $\langle T_2, T_3, T_1 \rangle$
 $\langle T_3, T_1, T_2 \rangle$
 $\langle T_3, T_2, T_1 \rangle$

6 Serial Schedule

▶ 2:01:20

Topic : Serializable Schedule

A Schedule is serializable Schedule if it is equivalent to a Serial Schedule.

(i) Conflict Serializability
(ii) View Serializability

Concurrent schedule

Serializable Schedule

Serial Schedule

Non - Serializable Schedule {Isolation failed}

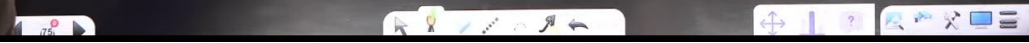
▶ 2:01:21



Topic : Serializability



- ❑ **Basic Assumption:** Each transaction preserves database consistency.
- ❑ Thus, serial execution of a set of transactions preserves database consistency.
- ❑ A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
 1. Conflict serializability
 2. view serializability



▶ 2:01:22



Topic : Serializability



- ❑ **Basic Assumption:** Each transaction preserves database consistency.
- ❑ Thus, serial execution of a set of transactions preserves database consistency.
- ❑ A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
 1. Conflict serializability
 2. view serializability



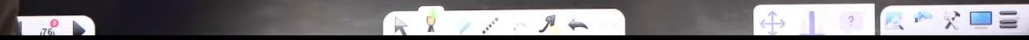
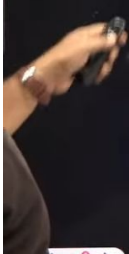
▶ 2:01:22



Topic : Conflict Serializability



- ❑ If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are conflict equivalent.
- ❑ We say that a schedule S is conflict serializable if it is conflict equivalent to a serial schedule.



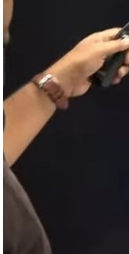
▶ 2:01:23



Topic : Conflict Serializability



- ❑ If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are conflict equivalent.
- ❑ We say that a schedule S is conflict serializable if it is conflict equivalent to a serial schedule.



▶ 2:01:23



Topic : Conflict Serializability (Cont.)



- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

Schedule 3

T_1	T_2
read (A)	
Write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 6

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

▶ 2:01:25



Topic : Conflict Serializability (Cont.)



- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

Schedule 3

T_1	T_2
read (A)	
Write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 6

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

▶ 2:01:25



2 mins Summary



✓ Topic

Dependency Preserving Decomposition

✓ Topic

Normal Form

Topic

Transaction Concept

ACID Property & Schedule

▶ 2:01:59